KNAPSACK PROBLEM VS CONTAINER LOADING

Dynamic Programming vs Greedy – Technical Report

1. Introduction

The 0/1 Knapsack Problem and Container Loading Problem both involve selecting items under a limited capacity. However, they represent different optimization strategies. In 0/1 Knapsack, each item has a weight and a value, and the objective is to maximize total value without exceeding capacity. In a simplified container-loading problem, the objective is commonly to maximize the amount of weight loaded, and a greedy strategy may select items according to a local rule such as largest weight first.

2. Problem Definition

Given n items with weights w[i], values v[i], and container capacity C, the 0/1 Knapsack objective is to choose each item at most once so that total weight is at most C and total value is maximum. The dynamic-programming recurrence is: dp[i][c] = max(dp[i-1][c], dp[i-1][c-w[i]] + v[i]) when w[i] <= c. This considers the two possibilities of excluding or including an item.

For container loading, a greedy method can sort items by weight in descending order and load an item whenever it fits. This is fast, but the locally best choice can prevent a better combination of smaller items from using the remaining capacity.

3. Example Input

The program uses the following illustrative data:

- Weights = [2, 3, 4, 5, 7]
- Values = [6, 5, 8, 9, 14]
- Container capacity = 10
- Each item can be selected at most once.

4. Java Implementation

```
import java.util.*;

public class KnapsackVsContainer {

    static class Item {
        int id;
        int weight;
        int value;

        Item(int id, int weight, int value) {
            this.id = id;
            this.weight = weight;
            this.value = value;
        }
    }

    // 0/1 Knapsack using Dynamic Programming
    static List<Integer> knapsackDP(Item[] items, int capacity) {
        int n = items.length;
```

```
    int[][] dp = new int[n + 1][capacity + 1];

    for (int i = 1; i <= n; i++) {
        for (int c = 0; c <= capacity; c++) {

            // Do not select item i
            dp[i][c] = dp[i - 1][c];

            // Select item i if it fits
            if (items[i - 1].weight <= c) {
                dp[i][c] = Math.max(
                    dp[i][c],
                    dp[i - 1][c - items[i - 1].weight]
                        + items[i - 1].value
                );
            }
        }
    }

    // Backtrack to find selected items
    List<Integer> selected = new ArrayList<>();
    int c = capacity;

    for (int i = n; i >= 1; i--) {
        if (dp[i][c] != dp[i - 1][c]) {
            selected.add(items[i - 1].id);
            c -= items[i - 1].weight;
        }
    }

    Collections.reverse(selected);
    return selected;
}

// Greedy container loading: largest weight first
static List<Integer> containerGreedy(Item[] items, int capacity) {

    Item[] sorted = items.clone();

    Arrays.sort(sorted, (a, b) ->
        Integer.compare(b.weight, a.weight)
    );

    List<Integer> selected = new ArrayList<>();
    int remaining = capacity;

    for (Item item : sorted) {
        if (item.weight <= remaining) {
            selected.add(item.id);
            remaining -= item.weight;
        }
    }

    return selected;
}

static int totalWeight(Item[] items, List<Integer> selected) {
    int total = 0;
```

```
    for (Item item : items) {
        if (selected.contains(item.id)) {
            total += item.weight;
        }
    }

    return total;
}

static int totalValue(Item[] items, List<Integer> selected) {
    int total = 0;

    for (Item item : items) {
        if (selected.contains(item.id)) {
            total += item.value;
        }
    }

    return total;
}

static void printResult(String method, Item[] items,
                        List<Integer> selected, int capacity) {

    int weight = totalWeight(items, selected);
    int value = totalValue(items, selected);

    double utilization = (weight * 100.0) / capacity;

    System.out.println("\n" + method);
    System.out.println("Selected items: " + selected);
    System.out.println("Total weight: " + weight);
    System.out.println("Total value: " + value);
    System.out.printf("Capacity utilization: %.2f%%%n", utilization);
}

public static void main(String[] args) {

    Item[] items = {
        new Item(1, 2, 6),
        new Item(2, 3, 5),
        new Item(3, 4, 8),
        new Item(4, 5, 9),
        new Item(5, 7, 14)
    };

    int capacity = 10;

    List<Integer> dpResult = knapsackDP(items, capacity);
    List<Integer> greedyResult =
        containerGreedy(items, capacity);

    printResult(
        "0/1 Knapsack - Dynamic Programming",
        items, dpResult, capacity
    );

    printResult(
        "Container Loading - Greedy",
```

```
            items, greedyResult, capacity
        );
    }
}
```

5. Expected Comparison

For the illustrative data, the DP algorithm searches combinations and guarantees the maximum total value. The greedy algorithm sorts by weight and takes the largest fitting items first. Because the greedy rule focuses only on the current item, it can select a combination that is feasible but not optimal in terms of value.

Capacity utilization and value are different measures. A route that fills more physical capacity is not automatically the best economic solution. In logistics, the value of the cargo, delivery priorities, fragility, volume, destination, and handling constraints can all matter.

6. Why Greedy Can Fail

Greedy algorithms make decisions using local information and normally do not reconsider earlier choices. This works for problems with the appropriate greedy-choice property, but general 0/1 Knapsack does not have that property. An item with a high weight may look attractive because it fills the container, while several smaller items may produce greater combined value.

Therefore, greedy is not guaranteed to find the globally optimal discrete solution. Dynamic programming handles the discrete trade-off explicitly by considering capacities and item combinations.

7. Computational Complexity

For n items and integer capacity C, the 0/1 Knapsack DP algorithm has time complexity O(nC) and space complexity O(nC) in the two-dimensional implementation. The space can be reduced to O(C) using a one-dimensional DP array.

The greedy approach requires sorting, which takes O(n log n), followed by a linear scan O(n). Thus its overall complexity is O(n log n), with O(n) additional storage when a sorted copy is maintained. Greedy is therefore substantially faster when a near-optimal or heuristic solution is acceptable.

8. Real-World Logistics Applications

- Warehouse packing: selecting valuable products within a fixed truck or container capacity.
- Cargo planning: prioritizing shipments while respecting weight limits.
- Delivery fleet loading: deciding which packages should be loaded when vehicle capacity is limited.
- Air freight: selecting high-priority or high-value cargo under weight constraints.
- Emergency logistics: maximizing the usefulness of supplies transported with limited vehicle space.
- Cloud and resource allocation: selecting tasks or workloads under limited resource capacity.

9. Solution Strategy Insights

Use dynamic programming when item selection is discrete, item values matter, the capacity is manageable, and an optimal solution is required. Use greedy methods when speed is more important, the problem has a provable greedy property, or the greedy result is being used as a practical heuristic.

For large real-world instances, hybrid strategies are often preferable. A greedy solution can provide a quick initial solution, after which local search, simulated annealing, genetic algorithms, or other metaheuristics can

improve the result. For very large capacity values, approximation algorithms or value-scaling techniques can also reduce computation.

10. Comparison Summary

Feature	0/1 Knapsack – DP	Container Loading – Greedy
Decision	Global combination search	Local selection rule
Optimality	Guaranteed optimum	Not guaranteed
Typical complexity	O(nC)	O(n log n)
Speed	Moderate to high cost	Fast
Best use	Exact discrete optimization	Fast practical loading
Main limitation	Capacity-dependent computation	Can miss better combinations

11. Conclusion

The comparison demonstrates the fundamental trade-off between optimality and computational speed. 0/1 Knapsack with dynamic programming evaluates the discrete inclusion/exclusion decisions and guarantees the best total value for the specified capacity. Greedy container loading is much faster because it makes simple local decisions, but it can sacrifice total value or utilization quality. In logistics, the choice should depend on problem size, decision time, business priorities, and the cost of obtaining a non-optimal loading plan.